Session 3: Pandas
Data Analysis with Python

ESS Programming with Python

Abate Alemayehu

Ethiopian Statistical Service (ESS)

June 15–30, 2026

Pandas Overview

Sessions

- Session 1: Pandas DataFrames (Slides 161–180)
- Session 2: Data Wrangling with Pandas (Slides 181–205)
- Session 3: Data Analysis with Pandas (Slides 206–225)
- Session 4: Visualizing Tabular Data (Slides 226–240)
- Session 5: Advanced Functions with Pandas (Slides 241–255)

Info

Pandas is the Excel-on-steroids for Python. Learn to clean, transform, and analyze real-world data.

Slide 161: What is Pandas?

Python Data Analysis Library

- Built on top of NumPy
- Provides high-level data structures
- Great for tabular data (like Excel or SQL)

Slide 162: Why Pandas?

Key Features

- Handles missing data gracefully
- Powerful data manipulation
- Labeled axes (row and column names)
- SQL-like operations (join, group by)
- Time series functionality

Slide 163: Installing and Importing

Installation

- Included in Anaconda
- `conda install pandas` or `pip install pandas`
- Import convention: `import pandas as pd`

```
1 import pandas as pd
```

Slide 164: Two Main Data Structures

Series and DataFrame

- **Series:** 1D labeled array (like a column)
- **DataFrame:** 2D labeled table (like Excel sheet)

Index	Series	DataFrame
0	10	Name, Age, City
1	20	Alice, 25, Addis
2	30	Bob, 30, Hawassa

Slide 165: Pandas Series

One-dimensional array with index labels

- Created from list: `pd.Series([1,2,3])`
- Custom index: `pd.Series([1,2,3], index=['a','b','c'])`
- Access by index or position

```
s = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
print(s['x'])  # 10
print(s[0])    # 10
```

Slide 166: Creating a DataFrame from Dictionary

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['Addis', 'Hawassa', 'Bahir Dar']
}
df = pd.DataFrame(data)
print(df)
```

	Name	Age	City
0	Alice	25	Addis
1	Bob	30	Hawassa
2	Charlie	35	Bahir Dar

Slide 167: Creating from List of Lists

```
data = [['Alice', 25, 'Addis'],
        ['Bob', 30, 'Hawassa'],
        ['Charlie', 35, 'Bahir Dar']]
df = pd.DataFrame(data, columns=['Name', 'Age', 'City'])
print(df)
```

Slide 168: Creating from NumPy Array

```
import numpy as np
arr = np.random.rand(5, 3)
df = pd.DataFrame(arr, columns=['A', 'B', 'C'])
print(df.head())
```

Slide 169: Viewing Data – head() and tail()

Quick Overview Methods

- `df.head()` → first 5 rows
- `df.head(10)` → first 10 rows
- `df.tail()` → last 5 rows

```
1 print(df.head())
2 print(df.tail(3))
```

Slide 170: Data Info – info()

DataFrame Information

- `df.info()`
- Shows: column names, non-null counts, data types
- Memory usage
- Very useful for data inspection

```
df.info()
# Output: RangeIndex, columns, non-null counts, dtypes
```

Slide 171: Summary Statistics – describe()

Statistical Summary

- `df.describe()` → for numeric columns
- Shows: count, mean, std, min, 25%, 50%, 75%, max
- `df.describe(include='all')` → includes categorical

```
1 print(df.describe())
2 print(df.describe(include='all'))
```

Slide 172: Accessing Columns

Column Selection

- `df['column_name']` → returns Series
- `df.column_name` → shorter but may conflict with methods
- Multiple columns: `df[['col1', 'col2']]` → returns DataFrame

```
df['Name']              # Series
df[['Name', 'Age']]     # DataFrame
```

Slide 173: Accessing Rows by Position – iloc[]

Integer Position-Based Indexing

- `df.iloc[0]` → first row
- `df.iloc[0:3]` → first 3 rows
- `df.iloc[0, 1]` → first row, second column
- Integer position only

```
print(df.iloc[0])       # First row
print(df.iloc[0:2])     # First two rows
print(df.iloc[0, 1])    # First row, second column
```

Slide 174: Accessing Rows by Label – loc[]

Label-Based Indexing

- `df.loc[0]` → row with index 0
- `df.loc[0:2]` → rows 0,1,2 (inclusive!)
- `df.loc[df.index[0]]` → first row by label

```
print(df.loc[0])       # Row with index label 0
print(df.loc[0:2])     # Rows 0, 1, 2 (inclusive)
```

Slide 175: Boolean Indexing in Pandas

Filtering with Conditions

- `df[df['Age'] > 30]` → rows where Age > 30
- Multiple conditions with `&` and `|`
- `df[(df['Age'] > 25) & (df['City'] == 'Addis')]`

```
# Age greater than 30
df[df['Age'] > 30]

# Age > 25 AND City is Addis
df[(df['Age'] > 25) & (df['City'] == 'Addis')]
```

Slide 176: Setting and Resetting Index

Index Operations

- `df.set_index('column_name')` → make column the index
- `df.reset_index()` → move index back to column
- `df.reset_index(drop=True)` → discard old index

```
1 df.set_index('Name', inplace=True)
2 df.reset_index(inplace=True)
```

Slide 177: Column Data Types

Data Types

- `df.dtypes` → data type of each column
- `df['col'].astype('int')` → convert type
- Common types: int64, float64, object (string), datetime64

```
print(df.dtypes)
df['Age'] = df['Age'].astype('float64')
```

Slide 178: Changing Column Names

Renaming Columns

- `df.columns` → get column names
- `df.columns = ['new1', 'new2', 'new3']` → rename all
- `df.rename(columns='old':'new')` → rename specific

```
1 df.columns = ['A', 'B', 'C']
2 df.rename(columns={'A': 'NewName'}, inplace=True)
```

Slide 179: DataFrame Shape and Dimensions

Shape Information

- `df.shape` → (rows, columns)
- `df.size` → total elements (rows × columns)
- `len(df)` → number of rows
- `df.ndim` → number of dimensions (always 2)

```
print(df.shape)    # (100, 5)
print(df.size)     # 500
print(len(df))     # 100
```

Slide 180: Code Example: Exploring a DataFrame

```python
import pandas as pd

# Create sample data
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age': [25, 30, 35, 28],
    'Salary': [50000, 60000, 70000, 55000]
})

print("First 2 rows:\n", df.head(2))
print("\nInfo:\n")
df.info()
print("\nDescribe:\n", df.describe())
print("\nPeople over 27:\n", df[df['Age'] > 27])
```

Slide 181: What is Data Wrangling?

Data Cleaning and Transformation

- Process of cleaning and transforming raw data
- Also called data munging
- Often 80% of data science work
- Essential for analysis

Slide 182: Handling Missing Data – Detection

Detecting Missing Values

- `df.isna()` → boolean mask of missing values
- `df.isnull()` → same as isna()
- `df.notna()` → opposite
- `df.isna().sum()` → count missing per column

```
1 print(df.isna().sum())  # Count missing per column
```

Slide 183: Dropping Missing Values – dropna()

Removing Missing Values

- `df.dropna()` → drop rows with any missing
- `df.dropna(axis=1)` → drop columns with any missing
- `df.dropna(thresh=2)` → keep rows with at least 2 non-NA
- `df.dropna(subset=['col1', 'col2'])` → check specific columns

```
1 df.dropna(inplace=True)
```

Slide 184: Filling Missing Values – fillna()

Replacing Missing Values

- `df.fillna(0)` → replace with 0
- `df.fillna(method='ffill')` → forward fill (use previous value)
- `df.fillna(method='bfill')` → backward fill
- `df.fillna(df.mean())` → fill with column mean

```
1 df['Age'].fillna(df['Age'].mean(), inplace=True)
```

Slide 185: Interpolating Missing Values

Interpolation

- `df.interpolate()` → linear interpolation
- `df.interpolate(method='polynomial', order=2)` → polynomial
- Works well for time series

```
df.interpolate(method='linear', inplace=True)
```

Slide 186: Adding New Columns

Creating Columns

- `df['new_col'] = values` → add column
- From existing: `df['Age_days'] = df['Age'] * 365`
- From condition: `df['Senior'] = df['Age'] > 60`

```
1 df['Age_Days'] = df['Age'] * 365
2 df['Senior'] = df['Age'] > 60
```

Slide 187: Deleting Columns and Rows

Removing Data

- `df.drop('column_name', axis=1)` → delete column
- `df.drop(['col1', 'col2'], axis=1)` → delete multiple
- `df.drop(index=0)` → delete row by index
- `inplace=True` to modify original

```
1 df.drop('Age_Days', axis=1, inplace=True)
2 df.drop(index=0, inplace=True)
```

Slide 188: Filtering Rows with Conditions

Conditional Filtering

- Single condition: `df[df['Age'] > 30]`
- Multiple AND: `df[(df['Age'] > 30) & (df['Salary'] > 50000)]`
- Multiple OR: `df[(df['City'] == 'Addis') | (df['City'] == 'Hawassa')]`

```
filtered = df[(df['Age'] > 25) & (df['Salary'] > 50000)]
```

Slide 189: The isin() Method

Filtering by List of Values

- df[df['City'].isin(['Addis', 'Hawassa', 'Dire Dawa'])]
- Opposite: df[df['City'].isin(['Addis'])]

```
cities = ['Addis', 'Hawassa', 'Dire Dawa']
df_filtered = df[df['City'].isin(cities)]
```

Slide 190: The between() Method

Range Filtering

- `df[df['Age'].between(25, 35)]`
- Inclusive by default

```
df_age_filtered = df[df['Age'].between(25, 35)]
```

Slide 191: Sorting Data – sort_values()

Sorting Rows

- `df.sort_values('Age')` → ascending (default)
- `df.sort_values('Age', ascending=False)` → descending
- Multiple columns: `df.sort_values(['Age', 'Salary'])`
- Different orders: `df.sort_values(['Age', 'Salary'], ascending=[True, False])`

```
1 df_sorted = df.sort_values('Age', ascending=False)
```

Slide 192: Sorting by Index

Index-Based Sorting

- `df.sort_index()` → sort by row index
- `df.sort_index(axis=1)` → sort columns alphabetically

```
df.sort_index(inplace=True)
df.sort_index(axis=1, inplace=True)
```

Slide 193: Removing Duplicates

Duplicate Handling

- `df.duplicated()` → boolean mask of duplicates
- `df.drop_duplicates()` → remove duplicate rows
- `df.drop_duplicates(subset=['Name'])` → check specific columns
- `df.drop_duplicates(keep='last')` → keep last occurrence

```
1 df.drop_duplicates(subset=['Name'], keep='first', inplace=True)
```

Slide 194: String Operations – str Accessor

String Methods

- `df['Name'].str.lower()` → convert to lowercase
- `df['Name'].str.upper()` → uppercase
- `df['Name'].str.contains('a')` → contains substring
- `df['Name'].str.len()` → string length

```
1 df['Name'] = df['Name'].str.upper()
2 mask = df['Name'].str.contains('A')
```

Slide 195: More String Operations

Advanced String Methods

- `df['Name'].str.strip()` → remove whitespace
- `df['Name'].str.replace(' ', '_')` → replace characters
- `df['Name'].str.split('_')` → split into list
- `df['Name'].str.extract(r'(\d+)')` → regex extraction

```
1 df['Clean_Name'] = df['Name'].str.strip().str.replace(' ', '_')
```

Slide 196: Working with Dates – to_datetime()

Date Conversion

- `pd.to_datetime(df['date_column'])` → convert to datetime
- Handles many formats automatically
- `errors='coerce'` → turn invalid into NaT

```
1 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

Slide 197: Date/Time Properties

Accessing Date Components

- `df['date'].dt.year` → year
- `df['date'].dt.month` → month (1-12)
- `df['date'].dt.day` → day of month
- `df['date'].dt.dayofweek` → Monday=0, Sunday=6
- `df['date'].dt.hour`, `.dt.minute`, `.dt.second`

```
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
```

Slide 198: Date Ranges and Offsets

Creating Date Ranges

- `pd.date_range('2024-01-01', periods=30, freq='D')` → daily
- `freq='H'` → hourly, `freq='M'` → month end, `freq='W'` → weekly

```
1 dates = pd.date_range('2024-01-01', periods=30, freq='D')
```

Slide 199: Applying Functions – apply()

Applying Functions to Data

- `df['col'].apply(lambda x: x*2)` → apply to column
- `df.apply(np.mean, axis=0)` → apply to each column
- `df.apply(np.mean, axis=1)` → apply to each row

```
1 df['Age_Double'] = df['Age'].apply(lambda x: x * 2)
```

Slide 200: Using applymap() for Element-wise Operations

Element-wise Operations

- `df.applymap(lambda x: x.upper())` → all string elements
- `df.applymap(lambda x: x**2)` → all numeric elements

```
df_strings = df.select_dtypes(include=['object']).applymap(str.
    upper)
```

Slide 201: The map() Method for Series

Value Mapping

- Replace values using dictionary
- df['col'].map('A':1, 'B':2, 'C':3)
- Also works with functions

```
mapping = {'A': 1, 'B': 2, 'C': 3}
df['Code_Num'] = df['Code'].map(mapping)
```

Slide 202: The replace() Method

Replacing Values

- `df.replace(0, 'zero')` → replace value
- `df.replace([1,2,3], ['one','two','three'])`
- `df.replace({1: 'one', 2: 'two'})`

```
1 df.replace({'A': 1, 'B': 2, 'C': 3}, inplace=True)
```

Slide 203: Renaming Columns with a Function

Programmatic Renaming

- `df.rename(columns=str.upper)` → uppercase all column names
- `df.rename(columns=lambda x: x.replace(' ', '_'))`

```
1 df.rename(columns=str.lower, inplace=True)
```

Slide 204: Working with Categorical Data

Categorical Data Type

- `df['col'].astype('category')` → convert to categorical
- Saves memory for repeated strings
- `df['col'].cat.categories` → get categories
- `df['col'].cat.codes` → integer codes

```
df['City'] = df['City'].astype('category')
print(df['City'].cat.categories)
```

Slide 205: Code Example: Data Wrangling I

```python
import pandas as pd
import numpy as np

# Load data with missing values
df = pd.read_csv('messy_data.csv')

# Clean column names
df.columns = df.columns.str.lower().str.replace(' ', '_')

# Handle missing values
df['age'].fillna(df['age'].median(), inplace=True)
df.dropna(subset=['name'], inplace=True)

# Filter and sort
```

Slide 205: Code Example: Data Wrangling II

```
15 df = df[df['age'].between(18, 100)]
16 df = df.sort_values('age', ascending=False)
17
18 # Create new column
19 df['age_group'] = pd.cut(df['age'],
20                          bins=[18, 30, 50, 100],
21                          labels=['Young', 'Middle', 'Senior'])
22
23 print(df.head())
```

Slide 206: The groupby() Method

Split-Apply-Combine Pattern

- Split data into groups based on criteria
- Apply function to each group
- Combine results

Split	**Group by category**
Apply	**Compute mean per group**
Combine	**Combine results**

Slide 207: Basic groupby

GroupBy Operations

- `df.groupby('column')` → creates GroupBy object
- `df.groupby('column').mean()` → mean per group
- `df.groupby('column').sum()` → sum per group
- `df.groupby('column').count()` → count per group

```
1 grouped = df.groupby('Category')
2 print(grouped.mean())
3 print(grouped.sum())
```

Slide 208: Grouping by Multiple Columns

Multi-Column Grouping

- `df.groupby(['col1', 'col2']).mean()`
- Creates MultiIndex
- Groups by unique combinations

```
result = df.groupby(['Region', 'Product'])['Sales'].sum()
print(result)
```

Slide 209: Aggregation Functions

Multiple Aggregations

- `group.agg('mean')` → same as .mean()
- `group.agg(['mean', 'std', 'count'])` → multiple at once
- Different per column: `group.agg({'col1':'mean', 'col2':'sum'})`

```
result = df.groupby('Category').agg({
    'Sales': 'sum',
    'Profit': 'mean',
    'Quantity': ['min', 'max']
})
```

Slide 210: Custom Aggregations

User-Defined Aggregations

- `group.agg(lambda x: x.max() - x.min())` → range
- `group.agg('describe')` → multiple statistics
- Can use named aggregations

```
def range_func(x):
    return x.max() - x.min()

result = df.groupby('Category')['Value'].agg(range_func)
```

Slide 211: transform() Method

Transform Operation

- Apply function that returns same shape as original
- Useful for adding group statistics back
- Example: `df['group_mean'] = df.groupby('group')['value'].transform('mean')`

```
df['Category_Mean'] = df.groupby('Category')['Value'].transform('mean')
```

Slide 212: filter() Method for Groups

Filtering Groups

- Keep groups that satisfy condition
- `df.groupby('group').filter(lambda x: len(x) > 5)` → groups with >5 rows
- `df.groupby('group').filter(lambda x: x['value'].mean() > 10)`

```
large_groups = df.groupby('Category').filter(lambda x: len(x) >= 10)
```

Slide 213: apply() on Groups

Flexible Group Application

- Apply any function to each group
- More flexible than agg/transform
- `df.groupby('group').apply(lambda x: x.sort_values('value').head(2))`

```
top2 = df.groupby('Category').apply(lambda x: x.nlargest(2, 'Sales'))
```

Slide 214: Pivot Tables

Creating Pivot Tables

- `pd.pivot_table(df, values='value', index='row', columns='col', aggfunc='mean')`
- Like Excel pivot tables
- aggfunc can be sum, count, len, etc.

```
pivot = pd.pivot_table(df,
                       values='Sales',
                       index='Region',
                       columns='Product',
                       aggfunc='sum')
```

Slide 215: Pivot Table Parameters

Pivot Table Options

- `fill_value=0` → replace missing with 0
- `margins=True` → add row/column totals
- `margins_name='Total'` → name for totals

```
pivot = pd.pivot_table(df,
                       values='Sales',
                       index='Region',
                       columns='Product',
                       aggfunc='sum',
                       fill_value=0,
                       margins=True,
                       margins_name='Total')
```

Slide 216: Cross Tabulation (crosstab)

Frequency Tables

- `pd.crosstab(df['col1'], df['col2'])` → frequency table
- `pd.crosstab(df['col1'], df['col2'], normalize='index')` → row percentages

```
cross = pd.crosstab(df['Category'], df['Region'])
cross_norm = pd.crosstab(df['Category'], df['Region'],
    normalize='index')
```

Slide 217: Merging DataFrames – merge()

Combining DataFrames

- Combine two DataFrames based on common column
- Like SQL JOIN
- `pd.merge(df1, df2, on='key_column')`

```
merged = pd.merge(df_orders, df_customers, on='CustomerID')
```

Slide 218: Types of Joins

Join Types

- `how='inner'` (default) → only matching keys
- `how='left'` → all keys from left
- `how='right'` → all keys from right
- `how='outer'` → all keys from both

```
1 inner = pd.merge(df1, df2, on='key', how='inner')
2 left = pd.merge(df1, df2, on='key', how='left')
3 outer = pd.merge(df1, df2, on='key', how='outer')
```

Slide 219: Merging on Different Column Names

Custom Merge Keys

- `pd.merge(df1, df2, left_on='col1', right_on='col2')`
- Merging on index: `left_index=True, right_index=True`

```
1 merged = pd.merge(df1, df2, left_on='ID', right_on='CustomerID'
    )
2 merged_idx = pd.merge(df1, df2, left_index=True, right_index=
    True)
```

Slide 220: Concatenation – concat()

Stacking DataFrames

- Stack DataFrames vertically or horizontally
- `pd.concat([df1, df2])` → vertical (more rows)
- `pd.concat([df1, df2], axis=1)` → horizontal (more columns)

```
vertical = pd.concat([df1, df2])
horizontal = pd.concat([df1, df2], axis=1)
```

Slide 221: Handling Indexes in concat()

Index Management

- `ignore_index=True` → reset index after concatenation
- `join='inner'` → only common columns
- `join='outer'` (default) → all columns

```
combined = pd.concat([df1, df2], ignore_index=True, join='inner')
```

Slide 222: Reshaping – melt()

Wide to Long Format

- Convert wide format to long format
- `pd.melt(df, id_vars=['id_col'], value_vars=['col1', 'col2'])`
- Opposite of pivot

```
long_df = pd.melt(df, id_vars=['Name'], value_vars=['Age', '
    Salary'])
```

Slide 223: Reshaping – pivot()

Long to Wide Format

- Convert long format to wide format
- `df.pivot(index='id_col', columns='variable', values='value')`

```
wide_df = long_df.pivot(index='Name', columns='variable',
    values='value')
```

Slide 224: Time Series Resampling

Resampling Time Series

- `df.set_index('date').resample('M').mean()` → monthly mean
- `resample('D')` → daily, `resample('W')` → weekly
- `resample('Y')` → yearly
- `ohlc()` → open, high, low, close for financial data

```
monthly = df.set_index('Date').resample('M').mean()
```

Slide 225: Code Example: Data Analysis I

```python
import pandas as pd
import numpy as np

# Sample sales data
df = pd.DataFrame({
    'Date': pd.date_range('2024-01-01', periods=100, freq='D'),
    'Product': ['A', 'B', 'C'] * 33 + ['A'],
    'Sales': np.random.randint(10, 100, 100),
    'Region': ['North', 'South'] * 50
})

# Group by analysis
monthly_sales = df.groupby([df['Date'].dt.month, 'Product'])['
    Sales'].sum()
```

Slide 225: Code Example: Data Analysis II

```
14 print("Monthly sales by product:\n", monthly_sales)
15
16 # Pivot table
17 pivot = pd.pivot_table(df, values='Sales', index='Region',
18                        columns='Product', aggfunc='mean')
19 print("\nAverage sales by region and product:\n", pivot)
20
21 # Time series resampling
22 df.set_index('Date').resample('W')['Sales'].sum().plot()
```

Slide 226: Pandas Plotting

Built-in Visualization

- Built-in plotting based on Matplotlib
- Quick and convenient for exploration
- `df.plot()` $\rightarrow$ line plot
- Returns matplotlib axes object

Slide 227: Line Plot

Line Plot

- `df.plot(kind='line')` or `df.plot()`
- Default for numeric data
- `df.plot(x='date', y='value')` → specify x and y

```
1 df.plot(x='Date', y='Sales')
2 plt.title('Sales Over Time')
3 plt.show()
```

Slide 228: Bar Plot

Bar Charts

- `df.plot(kind='bar')` → vertical bars
- `df.plot(kind='barh')` → horizontal bars
- `df['column'].value_counts().plot(kind='bar')` → for frequencies

```
df.groupby('Product')['Sales'].sum().plot(kind='bar')
plt.show()
```

Slide 229: Histogram

Distribution Plots

- `df.plot(kind='hist')` or `df.hist()`
- `bins=20` → number of bins
- `alpha=0.5` → transparency for overlaying

```
1 df['Sales'].plot(kind='hist', bins=30, alpha=0.7)
2 plt.show()
```

Slide 230: Box Plot

Box Plot

- `df.plot(kind='box')` or `df.boxplot()`
- Shows median, quartiles, outliers
- Good for comparing distributions

```
1 df.boxplot(column='Sales', by='Product')
2 plt.show()
```

Slide 231: Scatter Plot

Scatter Plot

- `df.plot(kind='scatter', x='col1', y='col2')`
- `c='col3'` → color by third variable
- `s=df['col4']*10` → size by fourth variable

```
1 df.plot(kind='scatter', x='Age', y='Salary', c='Sales',
      colormap='viridis')
2 plt.show()
```

Slide 232: Area Plot

Area Plot

- `df.plot(kind='area')`
- Stacked by default
- `stacked=False` for unstacked

```
1 df.set_index('Date')['Sales'].plot(kind='area')
2 plt.show()
```

Slide 233: Pie Chart

Pie Chart (Use Sparingly)

- `df['column'].value_counts().plot(kind='pie')`
- Use sparingly (hard to compare)
- `autopct='%1.1f%%'` → show percentages

```
1 df['Product'].value_counts().plot(kind='pie', autopct='%1.1f%%'
    )
2 plt.show()
```

Slide 234: Density Plot (KDE)

Kernel Density Estimation

- `df.plot(kind='kde')` or `df.plot.density()`
- Smooth version of histogram
- Good for comparing distributions

```
df['Sales'].plot(kind='kde')
plt.show()
```

Slide 235: Customizing Pandas Plots

Plot Customization

- `title='My Plot'` → title
- `xlabel='X axis'`, `ylabel='Y axis'`
- `legend=True/False`
- `figsize=(10,6)`

```
df.plot(figsize=(10, 6), title='Sales Analysis',
        xlabel='Date', ylabel='Sales', legend=True)
plt.show()
```

Slide 236: Subplots from DataFrame

Multiple Subplots

- `df.plot(subplots=True)` → separate subplot per column
- `layout=(2,3)` → 2 rows, 3 columns
- `sharex=True, sharey=True` → shared axes

```
df[['A', 'B', 'C']].plot(subplots=True, layout=(1, 3), figsize
    =(12, 4))
plt.tight_layout()
plt.show()
```

Slide 237: Plotting with GroupBy

Grouped Plots

- `df.groupby('group')['value'].plot(legend=True)`
- `df.groupby('group')['value'].mean().plot(kind='bar')`

```
df.groupby('Product')['Sales'].sum().plot(kind='bar')
plt.show()
```

Slide 238: Saving Pandas Plots

Saving Figures

- Get matplotlib axes: `ax = df.plot()`
- Save: `fig = ax.get_figure(); fig.savefig('plot.png')`
- Or: `df.plot().figure.savefig('plot.png')`

```
1 ax = df.plot()
2 ax.figure.savefig('my_plot.png', dpi=150)
```

Slide 239: Combining with Matplotlib

Advanced Customization

- Pandas plot returns axes object
- Use matplotlib functions for fine-tuning
- `ax = df.plot()` then `ax.set_ylabel('Custom label')`

```
ax = df.plot()
ax.set_ylabel('Custom Y Label')
ax.set_xlabel('Custom X Label')
ax.grid(True, alpha=0.3)
plt.show()
```

Slide 240: Code Example: Visualization I

```
import pandas as pd
import numpy as np

# Sample data
df = pd.DataFrame({
    'A': np.random.randn(1000),
    'B': np.random.randn(1000) + 2,
    'C': np.random.randn(1000) - 1
})

# Multiple plots
df.hist(bins=30, alpha=0.5, figsize=(10, 6))
plt.suptitle('Histograms')

```

Slide 240: Code Example: Visualization II

```
15 plt.figure()
16 df.plot(kind='box', figsize=(8, 6), title='Box Plot of
       Distributions')
17
18 plt.figure()
19 df.plot(kind='density', figsize=(10, 6), title='Density Plot')
20
21 # Bar plot of value counts
22 plt.figure()
23 df['A'].apply(lambda x: '>0' if x > 0 else '<=0').value_counts
       ().plot(kind='bar')
24 plt.title('Positive vs Negative Values')
25 plt.show()
```

Slide 241: Window Functions – Rolling

Rolling Windows

- `df['col'].rolling(window=7).mean()` → 7-day moving average
- `rolling(window=30).std()` → rolling standard deviation
- `rolling(window=5).sum()` → rolling sum

```
df['MA7'] = df['value'].rolling(window=7).mean()
df['volatility'] = df['value'].rolling(window=30).std()
```

Slide 242: Expanding Windows

Expanding Windows

- `df['col'].expanding().mean()` → cumulative mean
- `expanding().sum()` → running total
- `expanding().min()` → running minimum

```
df['cumsum'] = df['value'].expanding().sum()
df['cummean'] = df['value'].expanding().mean()
```

Slide 243: Exponential Weighted Functions

EWM (Exponential Weighted Moving)

- `df['col'].ewm(span=30).mean()` → exponentially weighted moving average
- More weight to recent observations
- `adjust=True` for bias adjustment

```
1 df['ewma'] = df['value'].ewm(span=30, adjust=True).mean()
```

Slide 244: Shift and Lag

Shifting Data

- `df['col'].shift(1)` → previous row (lag)
- `df['col'].shift(-1)` → next row (lead)
- `df['diff'] = df['col'] - df['col'].shift(1)` → difference

```
df['prev_value'] = df['value'].shift(1)
df['next_value'] = df['value'].shift(-1)
df['change'] = df['value'] - df['value'].shift(1)
```

Slide 245: Percent Change

Percentage Change

- `df['col'].pct_change()` → percentage change from previous
- `pct_change(periods=7)` → weekly percentage change
- `pct_change(periods=252)` → yearly for daily data

```
df['pct_change'] = df['value'].pct_change()
df['weekly_return'] = df['value'].pct_change(periods=7)
```

Slide 246: Rank and Quantile

Ranking and Quantiles

- `df['col'].rank()` → rank values (1 to N)
- `df['col'].rank(pct=True)` → percentile rank
- `df['col'].quantile([0.25, 0.5, 0.75])` → quartiles

```
df['rank'] = df['value'].rank()
df['percentile'] = df['value'].rank(pct=True)
print(df['value'].quantile([0.25, 0.5, 0.75]))
```

Slide 247: The cut() Function for Binning

Equal-Width Binning

- `pd.cut(df['age'], bins=[0,18,30,50,100], labels=['Child','Young','Adult','Senior'])`
- Creates categorical column
- `pd.cut(df['age'], bins=4)` → equal width bins

```
df['age_group'] = pd.cut(df['age'],
                         bins=[0, 18, 30, 50, 100],
                         labels=['Child', 'Young', 'Adult', '
                            Senior'])
```

Slide 248: The qcut() Function for Quantile Binning

Quantile-Based Binning

- `pd.qcut(df['value'], q=4, labels=['Q1','Q2','Q3','Q4'])`
- Equal number of observations per bin
- Good for creating ranks

```
df['quartile'] = pd.qcut(df['value'], q=4, labels=['Q1', 'Q2',
    'Q3', 'Q4'])
```

Slide 249: Memory Optimization

Reducing Memory Usage

- `df.info(memory_usage='deep')` → check memory
- Downcast integers: `df['col'] = pd.to_numeric(df['col'], downcast='integer')`
- Use categorical for low-cardinality strings
- `df = df.astype({col: 'float32' for col in float_cols})`

```
1 df['int_col'] = pd.to_numeric(df['int_col'], downcast='integer'
      )
2 df['cat_col'] = df['cat_col'].astype('category')
```

Slide 250: Working with Large Datasets – Chunking

Processing Data in Chunks

- `pd.read_csv('large.csv', chunksize=10000)` → iterator
- Process in chunks to avoid memory errors

```
chunk_size = 10000
for chunk in pd.read_csv('large.csv', chunksize=chunk_size):
    # Process each chunk
    process(chunk)
```

Slide 251: Query Method for Complex Filtering

SQL-like Query Syntax

- `df.query('Age > 30 and City == "Addis"')`
- More readable for complex conditions
- Can use variables with @: `df.query('Age > @threshold')`

```
threshold = 30
filtered = df.query('Age > @threshold and City == "Addis"')
```

Slide 252: eval() Method for Expressions

Evaluating Expressions

- `df.eval('Total = Price * Quantity')` → create column
- `df.eval('Price > 100 and Quantity > 0')` → boolean expression
- Faster than Python eval for large DataFrames

```
df.eval('Total = Price * Quantity', inplace=True)
high_value = df.eval('Price > 100 and Quantity > 0')
```

Slide 253: Pipe Method for Method Chaining

Function Chaining

- Pass DataFrame through functions
- `df.pipe(func1).pipe(func2, arg=value)`
- Clean alternative to nested functions

```
def clean_data(df):
    return df.dropna().query('Age > 0')

def add_features(df):
    df['Age_Squared'] = df['Age'] ** 2
    return df

result = df.pipe(clean_data).pipe(add_features)
```

Slide 254: Working with MultiIndex

Hierarchical Indexing

- `df.set_index(['col1', 'col2'])`
- Access: `df.loc[('value1', 'value2')]`
- Cross-section: `df.xs('value1', level=0)`

```
df_multi = df.set_index(['Region', 'Product'])
print(df_multi.loc[('North', 'A')])
print(df_multi.xs('North', level=0))
```

Slide 255: Code Example: Advanced Operations I

```
import pandas as pd
import numpy as np

# Time series analysis
dates = pd.date_range('2024-01-01', periods=365, freq='D')
df = pd.DataFrame({'value': np.random.randn(365).cumsum()},
    index=dates)

# Rolling statistics
df['MA7'] = df['value'].rolling(7).mean()
df['MA30'] = df['value'].rolling(30).mean()
df['volatility'] = df['value'].rolling(30).std()

# Binning
```

Slide 255: Code Example: Advanced Operations II

```
14 df['trend'] = pd.cut(df['value'], bins=10, labels=[f'Bin{i}'
       for i in range(10)])
15
16 # Efficient query
17 rising = df.query('value > value.shift(1)')
18 print(f"Days with increase: {len(rising)}")
19
20 # Display results
21 print(df.head())
```

End of Session 3

Questions?

Topics Covered:

- Pandas DataFrames (Creation, Access, Manipulation)
- Data Wrangling (Missing Data, Filtering, String Ops)
- Data Analysis (GroupBy, Pivot Tables, Merging)
- Visualization (Line, Bar, Hist, Box, Scatter)
- Advanced Functions (Rolling, Shift, Query, MultiIndex)